10.1 forcats: Factors

The level order is what tables and plots follow. Reorder, rename, collapse rare levels, and drop the ones a filter left behind.

A factor is a categorical vector: integer codes plus a `levels` attribute. The codes are what sits in the column. The levels are the labels, **in an order you choose**. Tables, legends, and axes follow that order. `forcats` comes with `library(tidyverse)`. Every helper starts with `fct_`.

A useful longer read: McNamara and Horton, Wrangling Categorical Data in R.

---

1. An integer plus labels

What prints is the label. Underneath, the values are `1`, `2`, `3`, … in the order of the levels. That is why `as.numeric()` on a factor of numbers returns the **index**.

```
library(tidyverse)

xf <- factor(c(10, 11, 13, 10))
xf
as.numeric(xf)
parse_number(as.character(xf))
```

```
## [1] 10 11 13 10
## Levels: 10 11 13
## [1] 1 2 3 1
## [1] 10 11 13 10
```

`as.character()` gets the labels back. `parse_number()` turns those labels into numbers. `levels(xf)[xf]` is the same recovery without converting to character first: a factor used as an index returns its labels.

`factor()` sorts the levels. `as_factor()` keeps the order the values first appear, which is what you want when the data is already in the right order.

```
factor(c("Wed", "Mon", "Fri", "Mon"))
as_factor(c("Wed", "Mon", "Fri", "Mon"))
```

```
## [1] Wed Mon Fri Mon
## Levels: Fri Mon Wed
## [1] Wed Mon Fri Mon
## Levels: Wed Mon Fri
```

Set `levels` yourself when the order is a real order, such as months or weekdays. A value that is not in `levels` becomes `NA`. An extra level stays in the attribute even when no row uses it.

```
weekdays <- c("Mon", "Tue", "Wed", "Thu", "Fri")
factor(c("Mon", "Sun"), levels = weekdays)
```

```
## [1] Mon  <NA>
## Levels: Mon Tue Wed Thu Fri
```

`read_csv()` does not make factors. A column stays character until you call `factor()`, `as_factor()`, or `col_factor()` inside `read_csv()`.

---

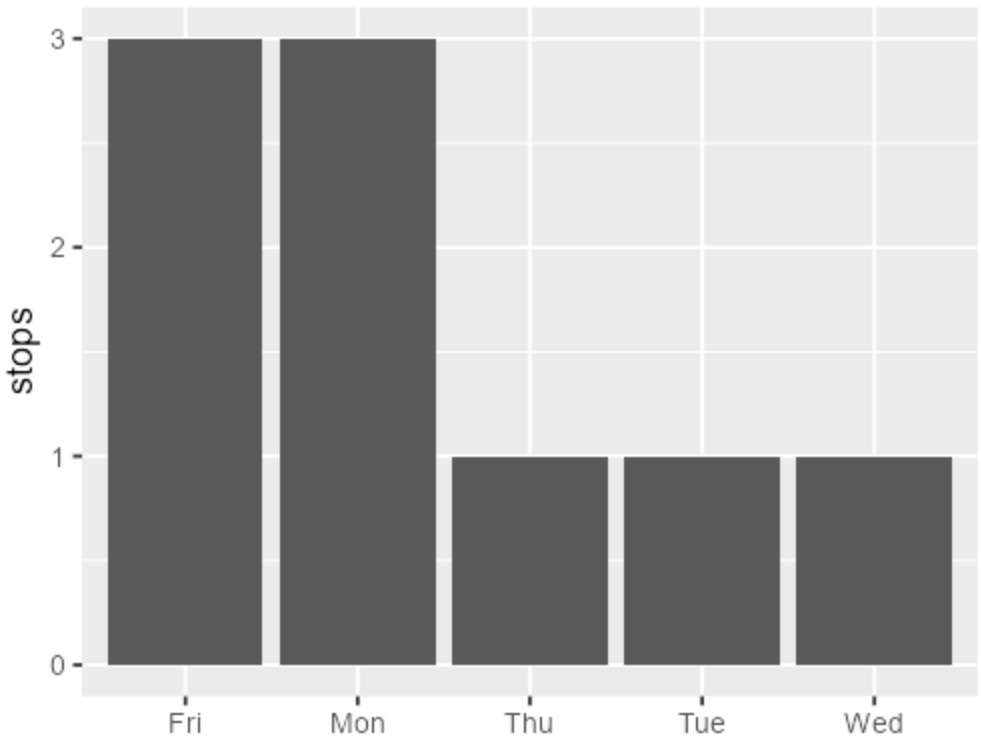
2. The axis follows the levels

Nine bus stops, stored as text. `factor()` with no `levels` argument sorts them alphabetically, so Friday comes before Monday.

```
weekdays <- c("Mon", "Tue", "Wed", "Thu", "Fri")
stops <- tibble(
  day = c("Wed", "Mon", "Fri", "Mon", "Tue", "Fri", "Fri", "Thu", "Mon")
)

stops |>
  mutate(day = factor(day)) |>
  count(day)
```

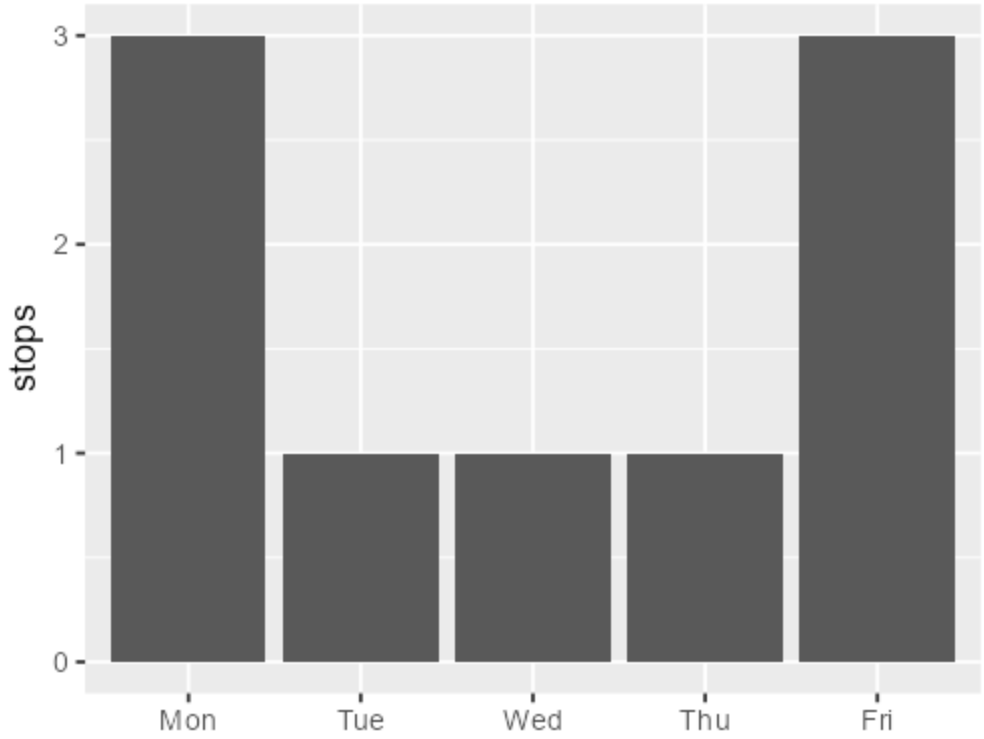
```
## # A tibble: 5 × 2
##   day       n
##   <fct> <int>
## 1 Fri       3
## 2 Mon       3
## 3 Thu       1
## 4 Tue       1
## 5 Wed       1
```



Pass the order you want. The counts do not change. The rows do not change. The axis does.

```
stops |>
  mutate(day = factor(day, levels = weekdays)) |>
  count(day)
```

```
## # A tibble: 5 × 2
##   day       n
##   <fct> <int>
## 1 Mon       3
## 2 Tue       1
## 3 Wed       1
## 4 Thu       1
## 5 Fri       3
```



On a vertical axis, ggplot draws the **first** level at the bottom. `fct_rev()` flips the current levels when you want the opposite end at the bottom.

---

3. `gss_cat`

`gss_cat` is a General Social Survey slice that ships with forcats. `?gss_cat`. The factors you will use are `marital`, `rincome`, `partyid`, and `relig`. `count()` follows the level order, so the first row is the first level, not the largest group.

```
count(gss_cat, partyid)
```

```
## # A tibble: 10 × 2
##    partyid               n
##    <fct>             <int>
##  1 No answer           154
##  2 Don't know            1
##  3 Other party         393
##  4 Strong republican  2314
##  5 Not str republican 3032
##  6 Ind,near rep       1791
##  7 Independent        4119
##  8 Ind,near dem       2499
##  9 Not str democrat   3690
## 10 Strong democrat    3490
```

`Independent` is the largest group and the seventh row, because that is where its level sits.

---

4. Which function

You want	Use
a fixed order (weekdays, months, a list you write)	`factor(x, levels = )` or `fct_relevel()`
levels sorted by a number (a mean, a rate)	`fct_reorder()`
most common level first	`fct_infreq()`
the current order flipped	`fct_rev()`
order of first appearance	`as_factor()` or `fct_inorder()`
new names, one level at a time	`fct_recode()`
several levels collapsed into one name	`fct_collapse()`
rare levels folded into Other	`fct_lump_n()` or `fct_lump_prop()`
unused levels removed after a filter	`fct_drop()`
a level added before any row has it	`fct_expand()`
missing values shown as their own level	`fct_na_value_to_level()`

---

5. Reorder by a summary

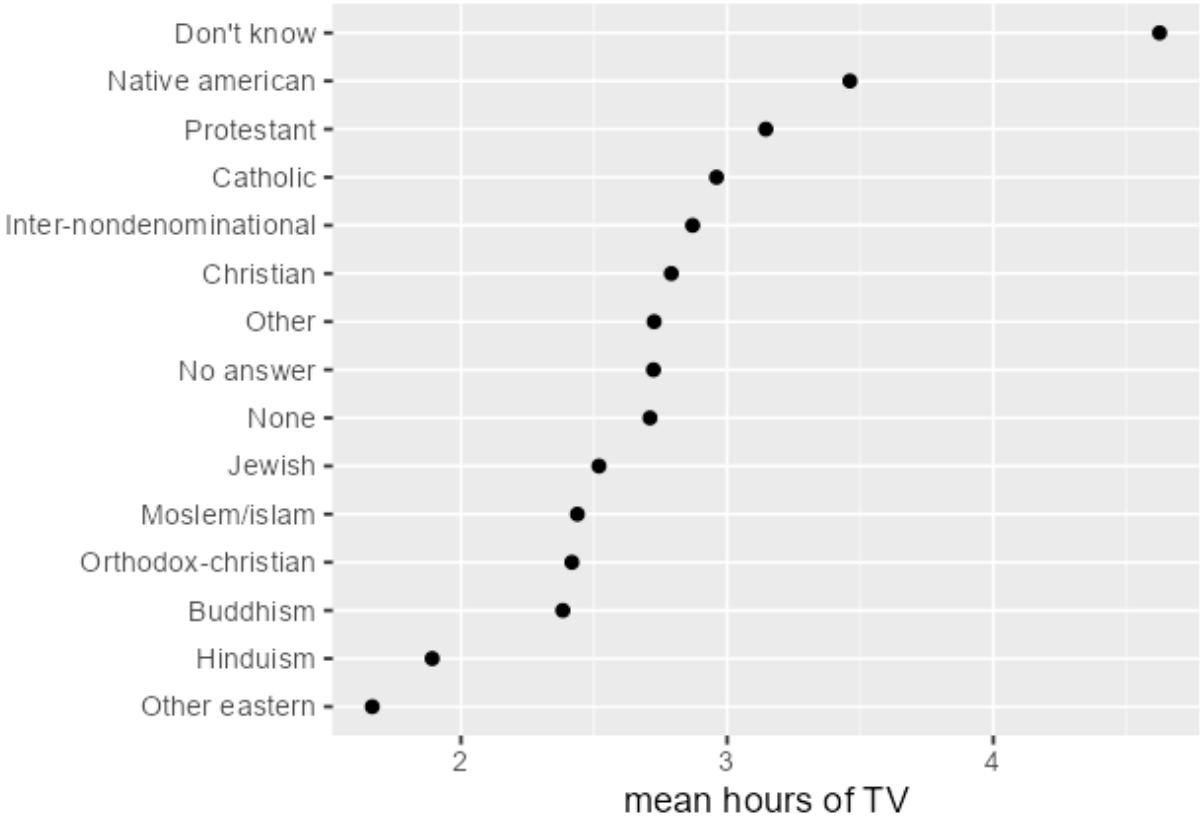
`fct_reorder(f, x)` sorts `f` by a summary of `x`. The default summary is the median. For a mean, pass the function and `.na_rm = TRUE` so missing numbers are dropped instead of warning.

Here the mean is computed first, one row per religion, and `fct_reorder()` sorts those rows. `arrange(relig)` prints them in the new level order. The plot uses the same order: least TV at the bottom, because that level is first.

```
tv <- gss_cat |>
  filter(!is.na(tvhours)) |>
  group_by(relig) |>
  summarize(hours = mean(tvhours), .groups = "drop") |>
  mutate(relig = fct_reorder(relig, hours)) |>
  arrange(relig)

tv
```

```
## # A tibble: 15 × 2
##    relig                  hours
##    <fct>                  <dbl>
##  1 Other eastern           1.67
##  2 Hinduism                1.89
##  3 Buddhism                2.38
##  4 Orthodox-christian      2.42
##  5 Moslem/islam            2.44
##  6 Jewish                  2.52
##  7 None                    2.71
##  8 No answer               2.72
##  9 Other                   2.73
## 10 Christian               2.79
## 11 Inter-nondenominational 2.87
## 12 Catholic                2.96
## 13 Protestant              3.15
## 14 Native american         3.46
## 15 Don't know              4.62
```



The same sort on the original rows, without summarizing first:

```
gss_cat |>
  mutate(relig = fct_reorder(relig, tvhours, mean, .na_rm = TRUE))
```

`fct_infreq()` sorts by how often each level occurs. Pipe it into `fct_rev()` when the longest bar should sit on the **right** of a vertical bar chart, or at the **top** of a horizontal one. `marital` below runs from rarest to most common.

```
gss_cat |>
  mutate(marital = marital |> fct_infreq() |> fct_rev()) |>
  count(marital)
```

```
## # A tibble: 6 × 2
##   marital           n
##   <fct>         <int>
## 1 No answer        17
## 2 Separated       743
## 3 Widowed        1807
## 4 Divorced       3383
## 5 Never married  5416
## 6 Married       10117
```

`fct_relevel()` moves named levels to the front and leaves the rest in their old order. A function in that spot is applied to the level names, so `sort` is alphabetical order.

```
gss_cat |>
  mutate(partyid = fct_relevel(partyid, "Independent")) |>
  count(partyid)

levels(fct_relevel(gss_cat$partyid, sort))
```

```
## # A tibble: 10 × 2
##    partyid                n
##    <fct>              <int>
##  1 Independent         4119
##  2 No answer            154
##  3 Don't know             1
##  4 Other party          393
##  5 Strong republican   2314
##  6 Not str republican  3032
##  7 Ind,near rep        1791
##  8 Ind,near dem        2499
##  9 Not str democrat    3690
## 10 Strong democrat     3490
##  [1] "Don't know"         "Ind,near dem"       "Ind,near rep"
##  [4] "Independent"        "No answer"          "Not str democrat"
##  [7] "Not str republican" "Other party"        "Strong democrat"
## [10] "Strong republican"
```

---

6. Rename and collapse

`fct_recode()` renames levels. The **new** name is on the left, the **old** name on the right, the same direction as `mutate()`. A name that matches nothing is left unchanged. `NULL = "old"` drops that level and turns its rows into `NA`.

```
gss_cat |>
  mutate(partyid = fct_recode(
    partyid,
    "Rep, strong" = "Strong republican",
    "Rep, weak"   = "Not str republican",
    "Dem, weak"   = "Not str democrat",
    "Dem, strong" = "Strong democrat"
  )) |>
  count(partyid)
```

```
## # A tibble: 10 × 2
##    partyid          n
##    <fct>        <int>
##  1 No answer      154
##  2 Don't know       1
##  3 Other party    393
##  4 Rep, strong   2314
##  5 Rep, weak     3032
##  6 Ind,near rep  1791
##  7 Independent   4119
##  8 Ind,near dem  2499
##  9 Dem, weak     3690
## 10 Dem, strong   3490
```

`fct_collapse()` is the same idea when several old levels share one new name.

```
gss_cat |>
  mutate(partyid = fct_collapse(
    partyid,
    other = c("No answer", "Don't know", "Other party"),
    rep   = c("Strong republican", "Not str republican"),
    ind   = c("Ind,near rep", "Independent", "Ind,near dem"),
    dem   = c("Not str democrat", "Strong democrat")
  )) |>
  count(partyid)
```

```
## # A tibble: 4 × 2
##   partyid     n
##   <fct>   <int>
## 1 other     548
## 2 rep      5346
## 3 ind      8409
## 4 dem      7180
```

Levels you do not name stay as they are.

---

7. Rare levels

A religion plot with fifteen labels is hard to read. `fct_lump_n(f, n)` keeps the `n` most common levels and folds the rest into `Other`. `Other` is an extra level, so `n = 5` produces **six** rows.

```
gss_cat |>
  mutate(relig = fct_lump_n(relig, n = 5)) |>
  count(relig, sort = TRUE)
```

```
## # A tibble: 6 × 2
##   relig          n
##   <fct>      <int>
## 1 Protestant 10846
## 2 Catholic    5124
## 3 None        3523
## 4 Other        913
## 5 Christian    689
## 6 Jewish       388
```

`fct_lump_prop(f, prop)` keeps levels that are at least that share of the rows. Ten percent leaves three religions, plus `Other`.

```
gss_cat |>
  mutate(relig = fct_lump_prop(relig, prop = 0.1)) |>
  count(relig, sort = TRUE)
```

```
## # A tibble: 4 × 2
##   relig          n
##   <fct>      <int>
## 1 Protestant 10846
## 2 Catholic    5124
## 3 None        3523
## 4 Other       1990
```

`fct_lump(f, n = 5)` is the older spelling of `fct_lump_n()`.

8. Unused levels and missing values

`filter()` removes rows. It does not remove levels. After keeping only Independents, `levels()` still lists all ten parties, and a bar chart still draws the empty ones.

```
party <- gss_cat |> filter(partyid == "Independent")
levels(party$partyid)
```

```
##  [1] "No answer"          "Don't know"         "Other party"
##  [4] "Strong republican"  "Not str republican" "Ind,near rep"
##  [7] "Independent"        "Ind,near dem"       "Not str democrat"
## [10] "Strong democrat"
```

`fct_drop()` removes levels that no remaining row uses.

```
party |>
  mutate(partyid = fct_drop(partyid)) |>
  count(partyid)
```

```
## # A tibble: 1 × 2
##   partyid         n
##   <fct>       <int>
## 1 Independent  4119
```

`fct_expand()` adds a level before any row has it. Lab questions that say "add a level" mean this, not a new observation.

```
f <- factor(c("A", "B", "A"), levels = c("A", "B", "C"))
fct_drop(f)

g <- factor(c("A", "B"))
fct_expand(g, "C", "D")
```

```
## [1] A B A
## Levels: A B
## [1] A B
## Levels: A B C D
```

`NA` in a factor is not a level, so it disappears from `count()` and from a bar chart. `fct_na_value_to_level()` turns those missing values into a real level. `fct_explicit_na()` is the older name.

```
h <- factor(c("A", NA, "B"))
fct_na_value_to_level(h, level = "(Missing)")
```

```
## [1] A         (Missing) B
## Levels: A B (Missing)
```

---

9. Ordered factors

An **ordered** factor says the levels are ranked: low < medium < high, or January < February. `factor(x, levels, ordered = TRUE)` builds one. `month(date, label = TRUE)` and `wday(date, label = TRUE)` in lubridate return ordered factors, which is why a month axis comes out in calendar order.

Party and religion are not ranks. Leave them unordered. Order those with `fct_relevel()` or `fct_reorder()` when the plot needs an order. An ordered factor also changes the contrasts in a model, so use it only when the ranking is real.

---

10. Practice

Use `gss_cat` unless the question says otherwise.

1. Rebuild the weekday factor from section 2 so the levels run Friday back to Monday.
2. Reorder `partyid` alphabetically.
3. Move the `rincome` level `"Not applicable"` to the front.
4. Abbreviate `marital` with `fct_recode()`: Never, Sep, Div, Wid, Mar.
5. Keep the four most common `relig` levels. How many rows does `count()` return, and why?
6. Keep the rows with `marital == "Married"`. Look at `levels()` before and after `fct_drop()`.

Solutions

```
stops |>
  mutate(day = factor(day, levels = weekdays) |> fct_rev()) |>
  count(day)

levels(fct_relevel(gss_cat$partyid, sort))

levels(fct_relevel(gss_cat$rincome, "Not applicable"))

gss_cat |>
  mutate(marital = fct_recode(
    marital,
    Never = "Never married",
    Div = "Divorced",
    Wid = "Widowed",
    Sep = "Separated",
    Mar = "Married"
  )) |>
  count(marital)

gss_cat |>
  mutate(relig = fct_lump_n(relig, n = 4)) |>
  count(relig, sort = TRUE)

married <- gss_cat |> filter(marital == "Married")
levels(married$marital)
levels(fct_drop(married$marital))
```

Friday-to-Monday is the weekday levels reversed, so the counts run Fri, Thu, Wed, Tue, Mon. Alphabetical `partyid` starts at `Don't know` and ends at `Strong republican`. `"Not applicable"` is the first `rincome` level; `No answer` and `Don't know` stay next.

The recode leaves `No answer` alone and renames the other five:

```
##   marital       n
## 1 No answer    17
## 2 Never      5416
## 3 Sep         743
## 4 Div        3383
## 5 Wid        1807
## 6 Mar       10117
```

`fct_lump_n(relig, n = 4)` returns **five** rows. The four kept levels are Protestant, Catholic, None, and Christian. `Other` is the fifth, and it is larger than Christian (1301 against 689), so a count sorted by `n` places `Other` above Christian.

After the filter, `levels()` still lists all six marital statuses. `fct_drop()` leaves only `Married`.